State-of-the-Art Survey: Qubit Calibration on Superconducting Quantum Processors

With Special Focus on Google Quantum AI — A Reproducibility-Oriented Review

Compiled March 14, 2026

Abstract. Calibration of superconducting transmon qubits is the rate-limiting step in scaling quantum processors toward fault-tolerant operation. This survey provides a comprehensive, reproduction-oriented review of the state of the art, with particular attention to the calibration stack developed by Google Quantum AI for the Sycamore and Willow processors. We cover the full calibration hierarchy — from single-qubit spectroscopy and gate tune-up through two-qubit gate optimization, readout calibration, leakage management, frequency-collision avoidance, and system-level benchmarking. For each stage we document the published protocols, identify the specific information gaps that prevent independent reproduction, and compare approaches across major platforms (Google, IBM, Rigetti, academic groups). We find that while the architectural framework is well described, critical implementation details — tolerance thresholds, optimizer internals, TLS-forecasting algorithms, and full calibration-graph specifications — remain undisclosed. We catalog over 60 primary references spanning 2014–2026 and distill ten open problems for the calibration community.

Contents

I. Introduction . . . 3
II. The Optimus Calibration Framework . . . 3
  II.A. Architecture and DAG Structure . . . 3
  II.B. Node Interaction Protocol . . . 3
  II.C. Graph Traversal Algorithms . . . 4
  II.D. Tolerances and Timeouts . . . 4
III. Single-Qubit Calibration . . . 4
  III.A. Spectroscopy and Basic Parameter Extraction . . . 4
  III.B. DRAG Pulse Calibration . . . 5
  III.C. Google's Single-Qubit Gate Performance . . . 5
IV. Two-Qubit Gate Calibration . . . 6
  IV.A. CZ Gate on Google Processors . . . 6
  IV.B. The Snake Optimizer for Frequency Configuration . . . 6
  IV.C. Cross-Resonance Gate Calibration (IBM Architecture) . . . 7
  IV.D. Parallel CZ Calibration at Scale . . . 8
V. Dispersive Readout Calibration . . . 8

V.A. Fundamentals . . . 8
V.B. Model-Based Readout Optimization (Google) . . . 8
V.C. Dynamic Dispersive Readout . . . 8
V.D. Readout Performance on Google Processors . . . 8
VI. Frequency-Collision Management . . . 9
VI.A. The Problem . . . 9
VI.B. Fabrication-Level Approaches . . . 9
VI.C. Google's Approach: Snake + TLS Forecasting . . . 9
VII. Leakage Detection and Removal . . . 10
VII.A. The Leakage Problem . . . 10
VII.B. Multi-Level Reset (MLR) . . . 10
VII.C. Data Qubit Leakage Removal (DQLR) . . . 10
VII.D. Advanced Leakage Mitigation . . . 10
VIII. Benchmarking and Verification . . . 11
VIII.A. Cross-Entropy Benchmarking (XEB) . . . 11
VIII.B. Randomized Benchmarking (RB) . . . 11
VIII.C. Character-Average Benchmarking (CAB) . . . 11
VIII.D. Gate Set Tomography (GST) . . . 11
VIII.E. Google's Error Budget . . . 11
IX. Drift Tracking and Real-Time Calibration . . . 12
IX.A. The Drift Problem . . . 12
IX.B. Millisecond-Scale FPGA-Based Calibration . . . 12
IX.C. Adaptive Relaxation Tracking . . . 12
X. Automation and Machine Learning . . . 13
X.A. Software Frameworks . . . 13
X.B. Machine-Learning Approaches . . . 13
X.C. Cirq Calibration API (Google) . . . 14
XI. Comprehensive Reproduction Gap Analysis . . . 14
XI.A. Summary of Gaps . . . 15
XI.B. Detailed Analysis of Critical Gaps . . . 15
XII. Comparison Across Platforms . . . 17
XIII. Google Processor Evolution: Calibration Milestones . . . 17
XIV. Open Problems and Future Directions . . . 17
XV. Conclusion . . . 18
References . . . 20

I. Introduction

Superconducting transmon qubits are the leading platform for near-term quantum computing, with processors now exceeding 100 physical qubits [1,2]. The central bottleneck to scaling is *calibration*: the process of experimentally determining and maintaining the $\mathcal{O}(100)$ control parameters per qubit that define gate operations, readout, and idle behavior. A single miscalibrated parameter can cascade through the processor, degrading logical error rates by orders of magnitude [3].

Google Quantum AI has published the most complete account of a production calibration stack, built around the **Optimus** framework [3] and the **Snake** frequency optimizer [4,5]. Their progression from the 53-qubit Sycamore processor [1] through the 72-qubit third-generation device [6] to the 105-qubit Willow chip [2] represents the most extensively documented calibration scaling trajectory in the field.

This survey is organized as follows. Section II reviews the Optimus calibration framework and its directed-acyclic-graph (DAG) architecture. Section III covers single-qubit calibration (spectroscopy, gate tune-up, DRAG). Section IV addresses two-qubit gate calibration for CZ, iSWAP, and cross-resonance gates. Section V treats dispersive readout optimization. Section VI discusses frequency-collision management and the Snake optimizer. Section VII covers leakage detection and removal. Section VIII reviews benchmarking and verification methods. Section IX addresses drift tracking and real-time calibration. Section X surveys automation and machine-learning approaches. Section XI synthesizes the identified reproduction gaps. Section XV concludes.

II. The Optimus Calibration Framework

II.A. Architecture and DAG Structure

The foundational organizational principle for Google's calibration stack is the **directed acyclic graph** (DAG) introduced by Kelly *et al.* [3] and protected by US Patent 9,940,212 [7]. Each calibration procedure is a *node* in the graph; directed edges encode dependencies. The canonical bootstrapping order is:

1. **Readout calibration** (root — no dependencies)
2. **Qubit frequency identification** (depends on readout)
3. **Rabi driving calibration** (depends on frequency — determines π-pulse amplitude)
4. **Single-qubit gate calibration** (X/Y amplitudes, phases, DRAG coefficient)
5. **Two-qubit gate calibration** (depends on single-qubit gates)
6. **System-level / algorithm-specific calibration**

Cyclic dependencies, where they arise, are broken by partitioning into *coarse*, *mid*, and *fine* calibration layers.

II.B. Node Interaction Protocol

Each node supports three methods, ordered by computational cost:

`check_state` (zero cost). A metadata-only check that passes if: (a) the most recent `check_data` or `calibrate` call is within the node's *timeout period*; (b) no unresolved calibration failures exist; (c) no dependency has been recalibrated since the last validation; and (d) all dependencies themselves pass `check_state`.

`check_data` (low cost). Runs a minimal experiment and classifies the result into one of three outcomes: *in spec* (parameter within tolerance), *out of spec* (detectable shift), or *bad data* (noise-like response indicating a deeper dependency failure).

`calibrate` (high cost). Acquires a full dataset, performs analysis, updates the parameter store, and verifies data quality. This is the *only* method that modifies stored parameters.

II.C. Graph Traversal Algorithms

`maintain(node)`: Recursively descends from the target node to the root. At each level it calls `check_state`, escalating to `check_data` and then `calibrate` as needed.

`diagnose(node)`: Triggered when `check_data` returns *bad data*. Bypasses `check_state` entirely and works from experimental data, investigating ancestors until the root cause is found and recalibrated.

II.D. Tolerances and Timeouts

Each node specifies a *tolerance* (e.g., a π-pulse must be within 10^{-4} radians of π rotation) and a *timeout period* reflecting the characteristic drift timescale. The supplementary material of the quantum-supremacy experiment confirms that **over 100 parameters per qubit** are maintained [8].

Reproduction Gap 1. The full DAG with concrete node names, analysis functions, tolerance values, and timeout periods has never been published. Only the abstract framework and a handful of illustrative examples are available. The Optimus codebase is proprietary.

III. Single-Qubit Calibration

III.A. Spectroscopy and Basic Parameter Extraction

The canonical single-qubit calibration sequence, common across all platforms [9,10], proceeds through five stages:

1. **Resonator spectroscopy.** Frequency sweep to identify readout resonators; extract bare resonator frequency at high power.
2. **Power-dependent spectroscopy (punchout).** Two-dimensional scan in amplitude and frequency; evaluate qubit viability through dispersive shift; classify each qubit as viable.
3. **Two-tone qubit spectroscopy.** Sweep drive frequency while monitoring readout; identify $|g\angle \to |e\angle$ transition and extract qubit frequency ω_q and anharmonicity α.

4. **Rabi oscillation.** Amplitude sweep at fixed duration to determine π-pulse and $\pi/2$-pulse amplitudes.
5. **Ramsey interferometry.** Two $\pi/2$ pulses with variable delay; extract T_2^* and fine frequency correction (typically ≤ 1 MHz).

T_1 (energy relaxation) is measured by exciting the qubit and varying the wait time before measurement. Typical values on current platforms range from 20 μs (Sycamore, 2019) to 68 $\pm$ 13 μs (Willow, 2024) [2].

III.B. DRAG Pulse Calibration

Derivative Removal by Adiabatic Gate (DRAG) is the universal technique for suppressing leakage to the $|2\angle$ state during fast single-qubit rotations. The in-phase component $I(t)$ uses a Gaussian or cosine envelope, while the quadrature component is $Q(t) = -\beta \cdot \dot{I}(t)$, with the DRAG coefficient $\beta = -1/(2\alpha)$ where α is the anharmonicity.

Calibration is performed via:
- **AllXY sequences**: pairs of gates (e.g., $R_x(\pi)$–$R_y(\pi/2)$) at different DRAG values [11].
- **Error amplification**: repeated $X_{\pi/2}$–$(X_\pi$–$Y_\pi)^n$–$Y_{\pi/2}$ sequences.

III.B.1. Advanced Pulse Envelopes

Hyppä *et al.* introduced **FAST DRAG** (Fourier Ansatz Spectrum Tuning), using a Fourier cosine-series envelope for broader spectral suppression, achieving leakage below 3×10^{-5} for gates as short as 6.25 ns — a 20× reduction over conventional DRAG [12]. They also proposed **HD DRAG** (Higher-Derivative) using higher-order time derivatives.

Malarchick systematically compared Gaussian, DRAG, and GRAPE (numerical optimal control) pulses [13]:
- For gate times ≥ 20 ns, properly calibrated DRAG achieves fidelities within 5×10^{-4} of the coherence limit.
- For gate times < 15 ns, GRAPE becomes necessary.
- DRAG exhibits superior robustness to frequency drift ($F_{\min} = 0.990$ vs. GRAPE's 0.931 over ± 5 MHz detuning).

III.C. Google's Single-Qubit Gate Performance

Parameter	**Sycamore (2019)**	**3rd Gen (2023)**	**Willow (2024)**
1Q gate fidelity	99.85%	—	99.97%
1Q gate duration	25 ns	$\sim$ 25 ns	$\sim$ 25 ns
T_1	$\sim$ 15–20 μs	$\sim$ 20 μs	68 $\pm$ 13 μs
$T_{2,\mathrm{CPMG}}$	—	—	89 μs
Anharmonicity	~ -200 MHz	—	—

Table 1: Evolution of single-qubit parameters across Google processor generations.

Reproduction Gap 2. Exact pulse envelopes (Gaussian vs. cosine, rise/fall times), DRAG coefficients, amplitude calibration precision targets, and the specific AllXY or error-amplification sequences used on Sycamore/Willow are not published. The gate duration of ~ 25 ns is stated but the envelope shape is internal.

IV. Two-Qubit Gate Calibration

IV.A. CZ Gate on Google Processors

Google's native two-qubit gate on Willow is the controlled-Z (CZ) gate, implemented by tuning neighboring qubits into the $|11\angle \leftrightarrow |02\angle$ avoided crossing. The qubit is swept through frequency space via a shaped flux pulse, accumulating a conditional phase of π.

The calibration involves:

1. **Chevron experiment**: 2D sweep of flux-pulse amplitude and duration to identify the $|11\angle$–$|02\angle$ resonance and optimal gate time.
2. **Conditional-phase verification**: Ramsey-like sequences confirming π phase accumulation when the control qubit is in $|1\angle$ and zero phase when in $|0\angle$.
3. **Dynamical-phase compensation**: Virtual Z rotations on both qubits to absorb single-qubit phase shifts.
4. **Coupler bias calibration**: Setting the tunable coupler's operating point to achieve the desired coupling strength while minimizing residual ZZ interaction during idle.

The Sycamore gate (a specific member of the f-sim family with $\theta = \pi/2$, $\varphi = \pi/6$) was native to the 2019 processor [1], with 12 ns duration and 99.64% average fidelity. Willow shifted to CZ ($\sim$ 26 ns, 99.88% fidelity) [2], sacrificing speed for compatibility with surface-code circuits.

IV.B. The Snake Optimizer for Frequency Configuration

At scale, calibrating individual two-qubit gates is insufficient — the *global frequency configuration* must be jointly optimized. Klimov *et al.* developed the **Snake optimizer** for this purpose [4,5].

For an N-qubit processor, the configuration space $\mathcal{F}$ has dimension $|\mathcal{F}| \sim 3N$ (idle frequencies + interaction frequencies for each edge), with a combinatorial search space of $k^{\mathcal{O}(N)}$ where $k \sim 100$ frequency options per gate.

The Snake decomposes this into local subproblems of dimension S^2 (scope parameter S). It traverses the processor graph, optimizing each element subject to constraints imposed by previously calibrated neighbors. The error model:

$$E(\mathcal{F} \mid A, D) = \sum_g \sum_m w_{g,m}(A) \cdot \varepsilon_{g,m}\big(\mathcal{F}_{g,m} \mid D\big)$$

captures four physical mechanisms:

- **Dephasing**: biases qubits toward maximum frequencies (flux-insensitive sweet spots).
- **Relaxation**: biases away from two-level-system (TLS) defect frequencies.

- **Stray coupling**: disperses frequencies to avoid collisions between parasitically coupled qubits.
- **Pulse distortion**: biases toward symmetric CZ resonance conditions.

The model uses $\sim$ 40,000 error components but only **16 trainable weights** for the full processor, trained on $\sim$ 6,500 cross-entropy benchmarks [5].

Configuration	**Cycle Error ($\times 10^{-3}$)**	**Runtime**
Unoptimized baseline ($N = 68$)	16.7 (median)	—
Snake $S = 1$ (local)	9.8	$\sim$ 6 s
Snake $S = 2$ (default)	7.2	$\sim$ 130 s
Snake $S = 4$	< 7.2	—
Snake $S = S_{\max}$ (global)	10.8	$\sim$ 6500 s

Table 2: Snake optimizer performance on 68-qubit processor. Note that global optimization ($S = S_{\max}$) is *worse* than intermediate scope, demonstrating the importance of constraint propagation [5].

Scaling projections for processors from $N = 17$ to $N = 1057$ (distance-3 to distance-23 surface codes) show that with "stitching" ($R = 4$ regions), a distance-23 logical qubit projects to cycle error of 6.3×10^{-3}.

Reproduction Gap 3. The *inner-loop optimizer* used within each Snake step (gradient-based? Nelder-Mead? CMA-ES?) is not specified. The specific frequency-separation rules for collision avoidance, the training procedure for the 16 weights, and the integration API between Snake and Optimus are not documented.

IV.C. Cross-Resonance Gate Calibration (IBM Architecture)

For completeness, we summarize the cross-resonance (CR) gate used by IBM, as it represents the major alternative two-qubit gate paradigm.

Sheldon *et al.* [14] established the systematic tuning procedure:
1. Optimize CR drive phase φ_0 to maximize ZX and zero ZY.
2. Align cancellation pulse phase $(\varphi_0 - \varphi_1)$ to suppress single-qubit IY.
3. Sweep cancellation amplitude until IX and IY vanish simultaneously.

The echoed CR protocol uses two CR pulses with opposite amplitude, accompanied by π rotations on the control qubit, creating constructive interference for the entangling ZX interaction while canceling unwanted terms (ZI, ZZ, IX). State-of-the-art fidelity: 99.40% at $\sim$ 280 ns (echo) [14] 99.78% at $\sim$ 180 ns with interference couplers.

Patterson *et al.* [15] combined continuous and repeated-gate Hamiltonian tomography for step-wise error reduction, achieving 97.0(7)% via interleaved randomized benchmarking. Multi-derivative DRAG applied to CR pulses further suppresses multiple leakage transitions simultaneously [16].

IV.D. Parallel CZ Calibration at Scale

Fan *et al.* demonstrated calibration of CZ gates across 52 qubits on a USTC processor [17], using a three-step fast-calibration procedure:

1. $|11\angle - |02\angle$ state exchange with superposed CZ layers ($N = 0-7$).
2. Conditional phase refinement via variable rotation X_β.
3. Dynamical phase compensation with Z_φ rotations on both qubits.

Non-overlapping gates are calibrated in parallel using Nelder-Mead optimization targeting "back probability" (return to $|00\angle$ after random two-qubit Cliffords). An inter-gate correlation metric $C_g = [F(U) - \prod F(U_i)]/\sqrt{F(U) \cdot \prod F(U_i)}$ quantifies crosstalk.

V. Dispersive Readout Calibration

V.A. Fundamentals

Superconducting qubits are measured via *dispersive readout*: the qubit is coupled to a readout resonator, and the qubit-state-dependent frequency shift χ (dispersive shift) allows discrimination via microwave transmission measurement.

Three parameters must be calibrated: the dispersive shift χ, the resonator linewidth κ, and the drive power. Sank *et al.* [18] developed a unified protocol measuring all three with few prerequisites, routinely deployed on chips with tens of qubits using automation software. Their models predict readout SNR to within 10% over 54-qubit devices. A critical finding: **resonator linewidth κ exhibits factor-of-2 variation** across devices, poorly controlled by fabrication.

V.B. Model-Based Readout Optimization (Google)

Bengtsson *et al.* [19] introduced model-based readout optimization achieving **1.5% measurement error per qubit** with 500 ns end-to-end duration, demonstrated simultaneously across 17 qubits. The approach uses a physical model of the dispersive readout to navigate the readout-power/speed/error tradeoff, suppressing measurement-induced state transitions that plague na"ive optimization.

V.C. Dynamic Dispersive Readout

Swiadek *et al.* [20] demonstrated dynamically reducing the qubit-resonator detuning during readout to increase χ, achieving **0.25% two-state readout error in 100 ns** — nearly quadrupling SNR by doubling the effective linewidth. This requires precise real-time flux control of the qubit frequency during the measurement window.

V.D. Readout Performance on Google Processors

Readout fidelity improved from 96.2% (Sycamore, 2019) to 99.5% (Willow, 2024), driven by:

- Optimized Purcell filter design reducing qubit decay through the readout channel.
- Model-based pulse shaping suppressing measurement-induced transitions.
- Improved single-shot IQ-plane discriminators.

Reproduction Gap 4. The readout model internals (exact Hamiltonian truncation level, pulse parametrization, optimizer used for readout pulse shaping) are described at the level of a PRL paper [19] but not open-sourced. Specific readout frequencies, powers, and integration windows for Willow are not published.

VI. Frequency-Collision Management

VI.A. The Problem

On processors with fixed-frequency or weakly-tunable transmons, qubit frequencies must be arranged to avoid *collisions* — degeneracies where parasitic interactions cause enhanced decoherence or gate errors. Key collision types include:

- $\omega_i = \omega_j$ (qubit-qubit resonance)
- $\omega_i = \omega_j + \alpha_j$ (straddling regime)
- $2\omega_i + \alpha_i = 2\omega_j + \alpha_j$ ($|02\angle - |20\angle$ resonance)

VI.B. Fabrication-Level Approaches

Osman *et al.* [21] demonstrated that fabricating larger Josephson junctions reduces wafer-level resistance standard deviation to 2%, achieving qubit frequency reproducibility with **40 MHz standard deviation (1%)** across 32 transmons. Simulations show their parametric-gate architecture scales to ~ 100 qubits with only 3 collisions on average.

IBM's **laser annealing** technique [22] selectively tunes individual transmon frequencies post-fabrication with **18.5 MHz precision** over hundreds of qubits, with no measurable coherence impact. This enabled a 65-qubit processor with median two-qubit gate fidelity of 98.7%.

The alternative **ABAA** (Alternating Bias Assisted Annealing) method achieves **7.7 MHz frequency precision** across 221 qubits [23], though junction-dependent relaxation remains the dominant imprecision source.

VI.C. Google's Approach: Snake + TLS Forecasting

Google addresses frequency collisions primarily through the Snake optimizer (Section IV.B), which builds collision avoidance into its error model as the "stray coupling" term. For Willow, an additional **TLS forecasting** algorithm predicts two-level-system defect frequencies to avoid coupling during initial calibration and over the experimental duration [2].

Reproduction Gap 5. The TLS forecasting algorithm is mentioned in the Willow paper but never described. Its inputs (historical TLS data? temperature dependence? spectral models?), prediction horizon, and accuracy are unknown. This is arguably the most significant undisclosed component for reproducing Willow-level performance.

VII. Leakage Detection and Removal

VII.A. The Leakage Problem

Transmon qubits are weakly anharmonic oscillators; the $|2\angle$ state lies only $|\alpha| \sim 200$ MHz above $|1\angle$. Gate operations — particularly two-qubit gates that traverse frequency space — can populate $|2\angle$ and higher levels. In quantum error correction, leaked population accumulates across cycles, causing correlated errors that degrade the code's performance far beyond what the leakage rate alone would suggest.

VII.B. Multi-Level Reset (MLR)

Google developed **multi-level reset** (MLR): an adiabatic swap between the qubit and readout resonator followed by a fast return, producing the ground state with **> 99% fidelity in 250 ns** from all relevant excited states [24].

VII.C. Data Qubit Leakage Removal (DQLR)

For the Willow below-threshold experiment, Google introduced **DQLR** [25], a two-step operation interleaved at the end of each QEC cycle:

1. **Leakage iSWAP gate**: splits the $|2\angle$ state (two excitations) into $|1\angle$ on two qubits, leaving lower states undisturbed.
2. **Measure qubit reset**: rapid reset to remove the transferred excitation.

Results: without DQLR, $\sim 1\%$ average leakage population; with DQLR, $\sim 0.1\%$ ($10\times$ reduction). The distance-7 code on Willow uses 4 additional leakage-removal qubits. DQLR increased the error-suppression factor Λ by 35% for distance-5 codes [2].

VII.D. Advanced Leakage Mitigation

Recent work achieves further improvements:

- **Active Leakage Cancellation** reduces leakage to 10^{-5} per gate [26].
- **Leakage Reduction Units (LRUs)** integrated with measurement achieve 99% leakage removal in 220 ns with zero additional time overhead [27].

Reproduction Gap 6. The leakage iSWAP gate calibration procedure — specifically, how the flux pulse is shaped to implement a state-selective iSWAP that acts on $|2\angle$ but not $|0\angle$ or $|1\angle$ — requires precise knowledge of the multi-level spectrum and coupler Hamiltonian, which is not documented step-by-step.

VIII. Benchmarking and Verification

VIII.A. Cross-Entropy Benchmarking (XEB)

XEB is Google's primary calibration-verification method [1]. It models the noisy output state as a depolarizing mixture:

$$\rho_U = f\ |\psi_U\rangle\langle\psi_U| + (1-f)\frac{I}{D}$$

where f is the circuit fidelity. The linear XEB estimator is:

$$f = \frac{\sum_U (m_U - u_U)(e_U - u_U)}{\sum_U (e_U - u_U)^2}$$

where m_U is the measured bitstring probability, e_U the ideal probability, and $u_U = 1/D$ the uniform baseline. XEB is used to:

- Characterize individual gate fidelities (isolated 2-qubit XEB).
- Detect systematic calibration errors via exponential fidelity decay.
- Verify parallel gate performance (simultaneous CZ + XEB).
- Benchmark full-system performance at scale.

VIII.B. Randomized Benchmarking (RB)

Standard RB applies uniformly random Clifford sequences of increasing length, appending an inversion Clifford, and fitting the exponential decay of return probability vs.
sequence length [11]. It is SPAM-insensitive and scales efficiently. **Interleaved RB** isolates the fidelity of a specific gate by alternating it with random Cliffords.

VIII.C. Character-Average Benchmarking (CAB)

Fan *et al.* [17] introduced CAB for scalable multi-qubit benchmarking, using alternate U and U^{-1} interleaved with random Paulis. Sample complexity is $\mathcal{O}(\varepsilon^{-2}\log\delta)$, independent of qubit count.

VIII.D. Gate Set Tomography (GST)

GST provides calibration-free, SPAM-independent characterization with Heisenberg scaling [28], but scales poorly beyond 2 qubits. Compressive and streaming variants are emerging for larger systems.

VIII.E. Google's Error Budget

The Willow below-threshold paper presents a detailed error budget decomposed across CZ gates, single-qubit gates, readout, and idling. CZ gates are the dominant error source, with correlated errors contributing $\sim$ 17%. Critically, the error model *overpredicts* Λ by $\sim$ 20%, indicating **unmodeled error sources** remain [2].

Reproduction Gap 7. The full error-budget decomposition methodology — how individual error contributions are extracted, what assumptions underlie the decomposition, and the source of the persistent 20% model–experiment gap — is not documented in sufficient detail to reproduce independently.

IX. Drift Tracking and Real-Time Calibration

IX.A. The Drift Problem

Qubit parameters fluctuate on timescales from milliseconds (TLS switching) to hours (thermal drift). Google recalibrates "every 4 experimental runs" on Willow, with stability demonstrated over 15 hours ($\Lambda = 2.18 \pm 0.07$ average, 2.31 ± 0.02 best) [2]. Kelly *et al.* [29] demonstrated **$\mathcal{O}(1)$-scalable in-situ calibration** during repetitive error detection, compensating for frequency drift on a 9-qubit system by optimizing gates in parallel.

IX.B. Millisecond-Scale FPGA-Based Calibration

Marciniak *et al.* [30] achieved a breakthrough in calibration speed using on-FPGA algorithms:

Primitive	**Time**	**Method**
T_1 estimation	9.8 ms	Analytical Decay Estimation (3-point)
Readout optimization	100 ms	Nelder-Mead (20 iterations)
Spectroscopy	39 ms	Golden-section search
π-pulse amplitude	1.1 ms	Sparse Phase Estimator
Clifford RB	107 ms	ADE on 3 sequence lengths
In-loop calibration	$\sim$ 31 ms	Combined frequency + amplitude

Table 3: FPGA-based calibration primitives from Marciniak *et al.* [30].

Over 6 hours of closed-loop operation, 74,525 recalibrations yielded a 6.4% reduction in average gate infidelity vs.
a static baseline.

IX.C. Adaptive Relaxation Tracking

Berritta *et al.* [31] demonstrated real-time Bayesian tracking of fluctuating T_1 values with ~ 2.2 μs per update on FPGA, completing full T_1 characterization in ~ 11 ms. They discovered TLS switching rates up to **10 Hz** — four orders of magnitude faster than previously reported — enabling real-time algorithm routing based on qubit quality.

Reproduction Gap 8. Google's recalibration cadence ("every 4 experimental runs") is stated without specifying what constitutes a "run," the total wall-clock time per calibration cycle, or the fraction of processor time consumed by calibration overhead.

X. Automation and Machine Learning

X.A. Software Frameworks

Several open-source frameworks now implement DAG-based calibration:

- **Qiskit Experiments** [11]: IBM's library offering `RoughFrequencyCal`, `FineDragCal`, `FineAmplitudeCal`, `HalfAngleCal`, and full RB/tomography suites.
- **Qibocal** [9]: Open-source framework supporting single-qubit spectroscopy through two-qubit CZ/iSWAP calibration, with CLI (`qq auto`) and Python eDSL interfaces.
- **QUAlibrate** [32]: Component-based DAG framework; demonstrated multi-qubit calibration in 140 seconds.
- **LabOne Q** [10]: Domain-specific language for QCCS hardware; characterized 26 of 28 qubits across seven chips in ~ 3.5 hours.
- **QubiC** [33]: Open-source FPGA-based system with multi-dimensional optimization for single- and two-qubit gates.

X.B. Machine-Learning Approaches

Pack *et al.* [34] benchmarked six gradient-free optimization algorithms for automated DRAG calibration, finding **CMA-ES** superior across all scenarios with effective noise resistance. Key insight: loss-function selection may be more critical than algorithm choice.

The Qruise platform creates **digital twins** of quantum devices via model learning, using Bayesian optimal experiment design to select the most informative measurements, then closing the loop with optimal control [35]. Rigetti demonstrated that AI-powered calibration (Quantum Elements + Qruise) achieved 99.9% single-qubit and 98.5% two-qubit fidelity on 9 qubits, replicating weeks of manual tuning [36].

AlphaQubit [37], while primarily a decoder, represents Google's ML approach to *device-adaptive* error correction: a transformer-based neural network fine-tuned on actual processor data, trained on 25 rounds and generalizing to 100,000+ rounds.

Recent work on **LLM-based agents** for calibration [38] demonstrates autonomous single- and two-qubit gate calibration on superconducting processors, though reliability for unsupervised operation is not yet established.

X.C. Cirq Calibration API (Google)

Google exposes calibration metrics through Cirq's programmatic interface, providing 21 metrics including single-qubit RB error, parallel readout errors, and two-qubit gate errors for both $\sqrt{\text{iSWAP}}$ and Sycamore gates. Access is via:

```
import cirq_google
cals = cirq_google.get_engine_calibration(
    PROCESSOR_ID, PROJECT_ID)
```

Historical and time-range queries are supported, enabling drift analysis.

Reproduction Gap 9. The Cirq API provides *read-only access* to calibration *results*, not to the calibration *procedures*. Users of Google's cloud quantum service cannot modify or inspect the calibration pipeline, run custom calibrations, or access the Optimus/Snake internals.

XI. Comprehensive Reproduction Gap Analysis

We synthesize the gaps identified throughout this survey into a structured catalog. These are organized by calibration stage and assessed for their impact on independent reproduction.

XI.A. Summary of Gaps

#	Gap Description	Impact
1	Full Optimus DAG (node names, analysis functions, tolerances, timeouts) is proprietary	Critical
2	Exact single-qubit pulse envelopes, DRAG coefficients, and amplitude-calibration targets for Sycamore/Willow	High
3	Snake optimizer inner-loop algorithm, frequency-separation rules, weight-training procedure, and Snake–Optimus integration API	Critical
4	Readout model internals: Hamiltonian truncation, pulse parametrization, optimizer choice	High
5	TLS forecasting algorithm: inputs, prediction method, horizon, accuracy	Critical
6	Leakage iSWAP calibration: multi-level spectrum requirements, flux-pulse shaping for state-selective operation	High
7	Error-budget decomposition methodology and source of the 20% model–experiment gap in Λ	Medium
8	Recalibration cadence: definition of "run," wall-clock time per cycle, calibration overhead fraction	Medium
9	Cirq API is read-only for metrics; calibration procedures are inaccessible to cloud users	High
10	Custom electronics: DAC specifications, signal-generation architecture, wiring, filtering, and attenuation	Critical

Table 4: Catalog of reproduction gaps in Google Quantum AI's published calibration materials. Impact assessed as Critical (prevents reproduction), High (requires significant reverse engineering), or Medium (workarounds exist).

XI.B. Detailed Analysis of Critical Gaps

XI.B.1. Gap 1: The Complete Calibration DAG

Kelly *et al.* [3] describe the *framework* but not the *content* of the calibration graph. For a 105-qubit processor with > 100 parameters per qubit, the full DAG likely contains $\mathcal{O}(10^4)$ nodes. Reconstructing this from scattered paper descriptions (spectroscopy, Rabi, Ramsey, DRAG, CZ chevron, XEB, etc.) is possible in principle but requires extensive experimental iteration to determine the correct dependency ordering and tolerance values.

What is available: the abstract framework, the bootstrapping order (readout $\rightarrow$ frequency $\rightarrow$ Rabi $\rightarrow$ 1Q gates $\rightarrow$ 2Q gates), and isolated examples of tolerance values (10^{-4} radians for π -pulses).

What is missing: the full node list, analysis functions per node, escalation logic, timeout values (characteristic drift timescales per parameter), and any conditional or context-dependent branches.

XI.B.2. Gap 3: Snake Optimizer Internals

The Snake's *outer loop* (graph traversal, scope parameter, constraint propagation) is well described [5]. The *inner loop* — the actual optimization algorithm applied to each S^2-dimensional subproblem — is not. Candidates include Nelder-Mead (used elsewhere in Google's stack), CMA-ES (recommended by Pack *et al.* [34]), or gradient-based methods with numerical derivatives.

The 16 trainable weights are trained on $\sim$ 6,500 XEB benchmarks, but the training procedure (loss function, optimizer, regularization, cross-validation) is not specified. The frequency-separation rules encoding collision avoidance are built into the "stray coupling" error term but not published as explicit constraints.

XI.B.3. Gap 5: TLS Forecasting

Two-level system (TLS) defects are the dominant source of frequency-dependent relaxation in transmon qubits [2]. The Willow paper states that frequencies are optimized to avoid TLS coupling and that a **forecasting** algorithm predicts TLS migration over the experimental duration. This suggests a model of TLS dynamics (activation, spectral diffusion, temperature dependence) that has not been published. Given that TLS behavior is stochastic and device-specific, this algorithm likely represents substantial proprietary knowledge.

XI.B.4. Gap 10: Custom Electronics

Google uses custom-designed room-temperature electronics (DACs, signal generators, bias tees) connected to the dilution refrigerator via filtered microwave lines. The specifications of these electronics — resolution, bandwidth, noise floor, timing jitter, and the specific filtering and attenuation scheme at each temperature stage — fundamentally limit achievable gate fidelities. While commercial alternatives exist (Zurich Instruments, Quantum Machines OPX, Keysight), replicating Google's performance may require matching their electronics specifications, which are not published in detail.

XII. Comparison Across Platforms

Aspect	Google	IBM	Rigetti	Academic
Native 2Q gate	CZ / Syc	CR (echoed)	CZ / iSWAP	Various
Best 2Q fidelity	99.88%	$\sim$ 99.5%	$\sim$ 98.5%	$\sim$ 99.8%
Calibration framework	Optimus (prop.)	Qiskit Exp.	Quil/proprietary	Qibocal, LabOne Q
Freq. optimizer	Snake (prop.)	Laser anneal	—	Manual / ABAA
Leakage removal	DQLR + MLR	DD-based	—	LRU variants
Automation level	Full auto	Semi-auto	AI-assisted	Semi-auto
Benchmarking	XEB	RB + QV	RB + QV	RB / GST
Open-source tools	Cirq (partial)	Qiskit (full)	pyQuil	Multiple
Recal. frequency	Every 4 runs	Hourly	—	Manual

Table 5: Comparison of calibration approaches across major superconducting qubit platforms.

XIII. Google Processor Evolution: Calibration Milestones

Metric	Sycamore (2019)	3rd Gen (2023)	Willow (2024)
Qubits	53 (54 fab.)	72	105
T_1	$\sim$ 15–20 μs	$\sim$ 20 μs	68 $\pm$ 13 μs
$T_{2,\mathrm{CPMG}}$	—	—	89 μs
1Q fidelity	99.85%	—	99.97%
2Q fidelity	99.64%	—	99.88%
Readout fidelity	96.2%	Improved	99.5%
Native gate	Sycamore	CZ	CZ
Gate time (2Q)	12 ns (Syc)	$\sim$ 26 ns	$\sim$ 26 ns
Cycle time	—	—	1.1 μs
QEC result	—	$\Lambda < 1$ (marginal)	$\Lambda = 2.14 \pm 0.02$
Logical qubit lifetime	—	—	2.4 $\times$ best physical
Calibration innovations	Optimus + XEB	Context-aware, parallel, TLS	Snake, DQLR, TLS forecast

Table 6: Evolution of Google Quantum AI processor calibration milestones.

XIV. Open Problems and Future Directions

Based on our survey, we identify ten open problems for the calibration community:

1. **The 20% model gap.** Google's best error model overpredicts Λ by $\sim 20\%$ [2]. Identifying the unmodeled error sources (non-Markovian noise, spatially correlated TLS, leakage–measurement crosstalk) is essential for pushing to higher code distances.
2. **Fat-tail T_1 distributions.** The worst 10% of qubits exhibit T_1 values 30–100× lower than the median, driven by TLS coupling. These outliers disproportionately limit logical performance. No predictive model exists for which qubits will be affected.
3. **Calibration time vs.**
   computation time. Current calibration overhead (~ 30 ms per primitive [30]) must decrease by ~ 3 orders of magnitude to enable kilohertz-rate recalibration during QEC. FPGA-based approaches are promising but have not been demonstrated at the 100+ qubit scale.
4. **Cross-platform reproducibility.** No standardized calibration protocol exists. Results from different platforms are not directly comparable due to differences in benchmarking methodology (XEB vs.
   RB vs.
   QV).
5. **Coupler calibration.** Tunable couplers add $\mathcal{O}(N)$ additional parameters (bias points, coupling-vs.-flux curves). Step-by-step coupler calibration procedures are largely absent from the literature.
6. **Correlated error characterization.** Standard benchmarks (RB, XEB) assume independent errors. Correlated errors — from crosstalk, TLS, or cosmic rays — require specialized protocols (simultaneous RB, crosstalk matrices) that scale poorly.
7. **Leakage–QEC interaction.** While DQLR reduces steady-state leakage to $\sim 0.1\%$, the *transient* leakage dynamics during a QEC cycle and their interaction with the decoder are not fully modeled.
8. **Automation generalization.** Current AI/ML calibration approaches (CMA-ES, RL, digital twins) are demonstrated on ≤ 10 qubits. Scaling to 100+ while maintaining the ability to recover from arbitrary failure modes remains open.
9. **Wiring and filtering at scale.** Each qubit requires 2–3 coaxial lines with precise attenuation and filtering at multiple temperature stages. Scaling to 1,000+ qubits requires fundamentally new wiring architectures, the calibration implications of which are unexplored.
10. **Real-time decoder–calibration co-design.** The Willow real-time decoder (63 μs latency at distance-5) must eventually incorporate calibration-state information to adapt decoding weights. This co-design space is largely unexplored.

XV. Conclusion

The calibration of superconducting quantum processors has evolved from manual, qubit-by-qubit tuning to automated, DAG-driven frameworks capable of maintaining $\mathcal{O}(10^4)$ parameters across 100+ qubits. Google Quantum AI's Optimus + Snake stack represents the most complete published

system, enabling the first demonstration of quantum error correction below the surface-code threshold on the 105-qubit Willow processor.

However, our analysis reveals that reproducing Google's results independently faces significant obstacles. Of the ten gaps cataloged in Table 4, four are assessed as *critical*: the complete calibration DAG, the Snake optimizer internals, the TLS forecasting algorithm, and the custom electronics specifications. These gaps are not incidental omissions but reflect the competitive dynamics of quantum hardware development, where calibration software is increasingly recognized as a key differentiator.

The path toward reproducibility passes through three developments: (1) open-source calibration frameworks (Qibocal, QUAlibrate) that implement the DAG architecture with community-contributed node libraries; (2) FPGA-based real-time calibration [30] that can close the 3-order-of-magnitude gap between current recalibration rates and QEC requirements; and (3) standardized benchmarking protocols that enable cross-platform comparison. Until these mature, independent groups seeking to approach Google-level calibration performance must expect to invest significant effort in reverse engineering and experimental optimization.

References

[1] F. Arute, K. Arya, R. Babbush, and others, Quantum supremacy using a programmable superconducting processor, Nature **574**, 10.1038/s41586-019-1666-5 (2019).

[2] Google Quantum AI, Quantum error correction below the surface code threshold, Nature 10.1038/s41586-024-08449-y (2024).

[3] J. Kelly, P. O'Malley, M. Neeley, H. Neven, and J. M. Martinis, Physical qubit calibration on a directed acyclic graph, Arxiv Preprint arXiv:1803.03226 (2018).

[4] P. V. Klimov, J. Kelly, J. M. Martinis, and H. Neven, The Snake Optimizer for Learning Quantum Processor Control Parameters, Arxiv Preprint arXiv:2006.04594 (2020).

[5] P. V. Klimov, A. Bengtsson, C. Quintana, and others, Optimizing quantum gates towards the scale of logical qubits, Nature Communications **15**, 10.1038/s41467-024-46623-y (2024).

[6] Google Quantum AI, Suppressing quantum errors by scaling a surface code logical qubit, Nature **614**, 10.1038/s41586-022-05434-1 (2023).

[7] J. S. Kelly, *Automatic Qubit Calibration*, US9940212B2 (2018).

[8] F. Arute and others, Supplementary information for 'Quantum supremacy using a programmable superconducting processor', Arxiv Preprint arXiv:1910.11333 (2019).

[9] A. Pasquale and others, Qibocal: an open-source framework for calibration of self-hosted quantum devices, Arxiv Preprint arXiv:2410.00101 (2024).

[10] Zurich Instruments, *Labone Q User Manual*, https://docs.zhinst.com/labone_q_user_manual/.

[11] IBM Research, *Qiskit Experiments: A Python Package to Characterize and Calibrate Quantum Computers*, https://qiskit-community.github.io/qiskit-experiments/.

[12] E. Hyppä and others, Reducing leakage of single-qubit gates for superconducting quantum processors using analytical control pulse envelopes, PRX Quantum **5**, arXiv:2402.17757 (2024).

[13] R. Malarchick, When does numerical pulse optimization actually help? Error budgets, robustness tradeoffs, and calibration guidance for transmon single-qubit gates, Arxiv Preprint arXiv:2511.12799 (2025).

[14] S. Sheldon, E. Magesan, J. M. Chow, and J. M. Gambetta, Procedure for systematically tuning up crosstalk in the cross resonance gate, Physical Review a **93**, arXiv:1603.04821 (2016).

[15] A. D. Patterson and others, Calibration of the cross-resonance two-qubit gate between directly-coupled transmons, Physical Review Applied **12**, arXiv:1905.05670 (2019).

[16] others, Experimental error suppression in Cross-Resonance gates via multi-derivative pulse shaping, Npj Quantum Information 10.1038/s41534-024-00863-4 (2024).

[17] D. Fan and others, Calibrating quantum gates up to 52 qubits in a superconducting processor, Npj Quantum Information **11**, arXiv:2505.22390 (2025).

[18] D. Sank, A. Opremcak, A. Bengtsson, and others, System Characterization of Dispersive Readout in Superconducting Qubits, Physical Review Applied **23**, arXiv:2402.00413 (2024).

[19] A. Bengtsson, A. Opremcak, M. Khezri, and others, Model-based Optimization of Superconducting Qubit Readout, Physical Review Letters **132**, arXiv:2308.02079 (2024).

[20] F. Swiadek and others, Enhancing Dispersive Readout of Superconducting Qubits Through Dynamic Control of the Dispersive Shift, PRX Quantum **5**, arXiv:2307.07765 (2024).

[21] A. Osman and others, Mitigation of frequency collisions in superconducting quantum processors, Physical Review Research **5**, arXiv:2303.04663 (2023).

[22] IBM Research, High-performance superconducting quantum processors via laser annealing of transmon qubits, Science Advances 10.1126/sciadv.abi6690 (2021).

[23] others, Post-fabrication frequency trimming of fixed-frequency transmon qubits using alternating bias assisted annealing, Arxiv Preprint arXiv:2407.06425 (2024).

[24] Google Quantum AI, Removing leakage-induced correlated errors in superconducting quantum error correction, Nature Communications **12**, arXiv:2102.06131 (2021).

[25] Google Quantum AI, Overcoming leakage in quantum error correction, Nature Physics 10.1038/s41567-023-02226-w (2023).

[26] others, Active leakage cancellation for superconducting qubits, Arxiv Preprint arXiv:2503.14731 (2025).

[27] others, Leakage reduction units integrated with measurement in superconducting qubits, Arxiv Preprint arXiv:2511.17460 (2025).

[28] E. Nielsen and others, Gate Set Tomography, Quantum **5**, 10.22331/q-2021-10-05-557 (2021).

[29] J. Kelly, R. Barends, A. G. Fowler, and others, Scalable in situ qubit calibration during repetitive error detection, Physical Review a **94**, arXiv:1603.03082 (2016).

[30] M. A. Marciniak and others, Millisecond-Scale Calibration and Benchmarking of Superconducting Qubits, Arxiv Preprint arXiv:2602.11912 (2026).

[31] F. Berritta and others, Real-time adaptive tracking of fluctuating relaxation rates in superconducting qubits, Arxiv Preprint arXiv:2506.09576 (2026).

[32] Quantum Machines, *Qualibrate: Open-Source Calibration Framework*, https://github.com/qua-platform/qualibrate.

[33] others, Automatic Qubit Characterization and Gate Optimization with QubiC, ACM Transactions on Quantum Computing 10.1145/3529397 (2022).

[34] K. Pack, S. Machnes, and F. K. Wilhelm, Benchmarking Optimization Algorithms for Automated Calibration of Quantum Devices, Arxiv Preprint arXiv:2509.08555 (2024).

[35] Qruise GmbH, *Qruise: Digital Twin and Closed-Loop Optimization for Quantum Devices*, https://www.qruise.com/.

[36] Rigetti Computing and Quantum Machines, *Quantum Machines and Rigetti Announce Successful AI-Powered Quantum Calibration Challenge*, https://investors.rigetti.com/news-releases/.

[37] Google DeepMind, *Alphaqubit: AI for Quantum Error Correction*, https://blog.google/technology/google-deepmind/alphaqubit-quantum-error-correction/.

[38] others, Automating quantum computing laboratory experiments with an agent-based AI framework, Cell Patterns 10.1016/j.patter.2025 (2025).